MedHire Admin Backend Handoff

Prepared: 9 October 2026 **Status:** Consolidated admin API contract. Confirm proposed endpoints, permissions, DTOs, and business rules against the backend before implementation.

Project locations: Candidate and recruiter frontend: D:\08-10-2026\medhire-main. Admin frontend: D:\08-10-2026\adminportal2026.

Base URL: Admin service paths below are relative to the configured API base. Configure one /api prefix consistently (for example VITE_API_BASE_URL=https://api.example.com/api) and avoid double-prefixing.

MedHire Admin Portal - Backend Handoff

Prepared: 9 October 2026

1. Scope and current status

This handoff covers the existing admin frontend: five authentication screens and Admin_06 through Admin_30. The admin frontend is integrated with API service adapters. Endpoint availability and behavior must be verified against the backend environment; the adapters show the intended request contract and do not imply that every proposed route is deployed.

Treat all HTTP endpoints below as proposed unless explicitly confirmed by the backend team. Field names describe frontend view models; backend DTOs may differ if the integration layer maps them explicitly.

- Revision 3 status: pages read and write through the API services in src/services (adminApi, adminCommerceService, adminOperationsService). There is no fixture fallback; an unavailable endpoint appears as an API error in the UI.
- Authentication fails closed: there are no working demo passwords or OTPs.
- Approvals, suspension, moderation, ticket, staff and settings actions send requests to the documented endpoints.
- Refund, receipt and invitation behavior depends on the backend confirming the proposed routes listed in section 23.
- Documents a hospital uploads are opened through short-lived signed links (section 25.3). Other previews, invoices and receipts depend on the proposed file endpoints.
- Loading and error states, silent session refresh, return to login after an expired session, and server-side sign-out are in place as of Revision 3. Server pagination still depends on the backend list endpoints.
- No Admin_31+ screens and no candidate- or recruiter-facing application are included in this portal.

2. Setup and integration files

Use Node.js 20.18+ and npm:

```
npm install

npm run dev
```

Open `/admin/dashboard` on the development server. Configure a local environment file using `.env.example`:

```
VITE_API_BASE_URL=http://localhost:5000/api
```

API paths below are relative to this base URL, including `/api`. Never store secrets in `VITE_` variables: they are bundled into the browser. Production hosting needs route rewrites to `index.html` for BrowserRouter.

| File | Integration responsibility |

| --- | --- |

| `src/App.jsx` | Routes and access guard; add server-backed session initialization |

| `src/services/api.js` | Axios client: 15-second timeout, JSON Accept header; no auth interceptors or explicit cross-origin cookie configuration yet |

| `src/services/authService.js` | Auth boundary; private adapter is `null`, with no exported installation function, so wiring requires a code change |

| `src/context/AdminDataContext.jsx` | Candidate, recruiter and plan state/mutations |

| `src/context/useOperationsState.js` | Jobs, notifications, tickets, staff and settings state/mutations |

| `src/services/adminCommerceService.js` | Local filters and plan/refund validation; not HTTP methods yet |

| `src/services/adminOperationsService.js` | Local filters, permission calculation and staff/settings validation |

| `src/data/*MockData.js` | Exact current view-model fixtures and relationships |

| `src/pages/admin/` | Several pages import fixtures directly; replacing context alone is insufficient |

| `src/utils/downloads.js` | CSV export and sample PDF generation |

| `src/components/layout/AdminProfile.jsx` | Hardcoded John Admin identity; replace with authenticated user |

3. Screen-to-API checklist

Browser routes are implemented. HTTP paths are proposals to confirm with the backend team. List APIs need server pagination, stable ordering and total counts.

| Frontend route | Current behavior | Proposed API / required capability |

| --- | --- | --- |

| `/login`, `/forgot-password`, `/verify-otp`, `/reset-password`, `/reset-success` | Authentication and recovery forms | Adapter contract in section 4 |

| `/admin/dashboard` | Summary cards, attention counts, chart and recent activity | `GET /admin/dashboard`: real aggregates, comparison periods and recent activity |

| `/admin/candidates` | Name search, account/resume filters, CSV | `GET /admin/candidates`: query, accountStatus, resumeStatus, page, pageSize |

| `/admin/candidates/:candidateId` | Profile, applications, uploads, unlocks, suspension | `GET /admin/candidates/{candidateId}`; `PATCH /admin/candidates/{candidateId}/status` with `{ accountStatus: 'Suspended' }` |

| `/admin/pending-resumes` | Review queue | Candidate list filtered by resumeStatus=Pending |

| `/admin/resume-review/:candidateId` | Preview/download, notes, approve/reject | Candidate detail and authorized document retrieval; `POST /admin/resumes/{resumeId}/review` with decision (APPROVED or REJECTED), notes, and reason (required when rejecting) |

| `/admin/recruiters` | Search, organization/verification filters, CSV | `GET /admin/recruiters`: query, organizationName, verificationStatus, page, pageSize |

| `/admin/recruiters/:recruiterId/verification` | Documents, checklist, approve/reject | `GET /admin/recruiters/{recruiterId}`; `POST /admin/organizations/{organizationId}/verify` with decision APPROVED or REJECTED and reason (required when rejecting) |

| `/admin/hospitals` | Read-only reporting, name/location search, plan filter, CSV | `GET /admin/hospitals`: query, planId, page, pageSize |

| `/admin/unlock-history` | Search, type/date filters, CSV | `GET /admin/unlock-history`: query, type, organizationId, candidateId, startDate, endDate, page, pageSize |

| `/admin/payments` | Search, status/date filters, metrics, CSV | `GET /admin/payments`: query, organizationId, status, startDate, endDate, page, pageSize; include aggregate totals |

| `/admin/payments/:transactionId` | Detail, plan snapshot, timeline, invoice, receipt, refund | `GET /admin/payments/{transactionId}`; `POST /admin/payments/{transactionId}/refund` with `{ amount: '100.00', reason: '...' }`; invoice/receipt contracts to agree |

| `/admin/plans` | Catalog, archive, credit ledger | `GET /admin/plans`; `PATCH /admin/plans/{planId}` with `{ active: false }`; ledger read contract to agree |

| `/admin/plans/new`, `/admin/plans/:planId/edit` | Create/edit plan | `POST /admin/plans`; `PATCH /admin/plans/{planId}` with editable fields only |

| `/admin/revenue` | Date filter, trend, plan breakdown, PDF summary | `GET /admin/revenue`: startDate, endDate, groupBy, organizationId |

| `/admin/jobs` | Search title/hospital, department/status filters, CSV | `GET /admin/jobs`: query, department, organizationId, status, page, pageSize |

| `/admin/jobs/:jobId` | Job detail, application counts, recruiter contact, flag/close | `GET /admin/jobs/{jobId}` returns the job detail (section 9). `PATCH /admin/jobs/{jobId}/status`: status, reason |

| `/admin/notifications` | Alerts, filtering, mark read/all read, preferences | `GET /admin/notifications`; per-user read-state and preference mutation contracts to agree |

| `/admin/support-tickets` | Search ID/subject/requester, category/priority filters, metrics | `GET /admin/support-tickets`: query, category, priority, status, requesterRole, page, pageSize; metrics |

| `/admin/support-tickets/:ticketId` | Conversation, reply, assignment, priority, status/history | `GET /admin/support-tickets/{ticketId}`; `POST /admin/support-tickets/{ticketId}/replies`: message; `PATCH /admin/support-tickets/{ticketId}/status`: status; assignment/priority contracts to agree |

| `/admin/staff` | Search, role filter, role edits, soft deactivation | `GET /admin/staff`: query, role, page, pageSize; `PATCH /admin/staff/{adminId}`; proposed `DELETE /admin/staff/{adminId}` means soft deactivation |

| `/admin/staff/invite` | Name, email, role and permission selection | `POST /admin/staff`; map UI name/role to previously proposed fullName/adminLevel; agree permissions payload |

| `/admin/staff/permissions` | Role cards and per-admin overrides | Role/permission reads and override mutation contract to agree |

| `/admin/settings` | Shared draft across General, Email, Security, Maintenance | `GET /admin/settings`; `PATCH /admin/settings` |

| `/admin/audit-logs` | Search, actor/action/date filters, detail, CSV | `GET /admin/audit-logs`: query, actorId, action, targetType, targetId, startDate, endDate, page, pageSize |

`/admin` redirects to the dashboard; `/admin/support` redirects to support tickets. Header search currently searches candidates only despite its broader placeholder. There is no hospital detail screen or notification broadcast composer; these are not required by the delivered UI.

4. Authentication and sessions

Exact normalized adapter interface currently expected by `authService.js`:

| Method | Input | Required result |

| --- | --- | --- |

| login | email, password, remember | `{ user: { id, role: 'ADMIN', ... }, sessionVerified: true }` after server verification |

| requestReset | email | Generic 202 { message } only. The UI starts a 10-minute code timer and a 60-second resend cooldown itself. |

| verifyOtp | email, code, purpose "password_reset" | resetToken (single use, short lived); an optional expiresAt in epoch milliseconds |

| resetPassword | email, resetToken, newPassword | 200 { message }; the UI treats any non-error response as success only after the password is committed |

Resend calls forgot-password again with the stored email after the cooldown. The email, reset token and session are held in memory and clear on refresh. Recovery route state or browser storage cannot authorize access. Revision 3: this replaces the earlier challengeId design; the admin flow now uses exactly the shared contract in the Auth Handoff, section 7.

Backend must validate credentials/admin access, derive identity from the session, issue and validate secure sessions, and supply session-lookup and logout/revocation operations. Define the lifetime associated with remember-me. Prefer HttpOnly Secure cookies with appropriate SameSite/CSRF protection; configure Axios credentials and allowed-origin CORS if cookies cross origins. Revision 3 defines these operations as POST /auth/refresh, POST /auth/logout and GET /auth/me (Auth Handoff section 6).

Recovery must return generic account responses, bind challenges to accounts, enforce OTP expiry/attempt limits/resend throttling, consume reset grants atomically, enforce password policy and invalidate affected sessions. Frontend timers are UX only. Normalize safe user-facing errors without exposing credentials or tokens.

UI validation: email maximum 254 characters; login password 1-128 characters, not whitespace-only; OTP exactly six digits; new password 8-128 characters including uppercase, number and special character; matching confirmation. Login does not impose new-password composition rules.

Mapping decision: authentication checks account role `ADMIN`; staff permissions use `SUPER_ADMIN`, `VERIFICATION_LEAD`, `SUPPORT_LEAD`, `FINANCE_ADMIN`. Agree a separate admin-level field or adapter mapping. `sessionVerified` must come from actual verification, never a client claim.

5. Current data models

These fields are normalized frontend requirements, not prescribed database tables. IDs must be stable server-issued identifiers. Related organization/recruiter names must be available in expanded DTOs or lookup APIs; one paginated list

cannot serve as a complete lookup table.

| Model | Current fields |

| --- | --- |

| Candidate | id, name, specialty, location, resumeStatus, accountStatus, degree, experience, hospital, email, phone, focus, license, submitted, verificationNotes, rejectionReason; applications[{role,hospital,date,status}], uploads[{name,date,status}], unlocks[{hospital,date,credits}] |

| Recruiter | id, name, organizationId, status, email, phone, registration, documents[]; decisions add reason |

| Hospital/organization | id, name, city, state, recruiters, unlocked, planId, status; recruiters/unlocked are numeric counts |

| Plan | Editable: name, type, price, credits, features, support, autoRenew, active. Server-owned: id, subscribers, revenue, used. Features is newline-separated text |

| Payment | id, recruiterId, organizationId, planId, planName, amount, credits, autoRenew, validity, date, status, method, reference; plan values are purchase snapshots |

| Unlock event | id, candidateId, name, recruiterId, organizationId, date, time, credits, type |

| Credit ledger | id, date, recruiterId, action, transfer, balance; transfer is signed |

| Job | id, title, organizationId, recruiterId, hospital, department, status, date, location, employment, experience, salary, overview, requirements[], moderationReason; applications{Applied,Shortlisted,Interviewed,Selected} are counts |

| Notification | id, type, title, message, read, createdAt; preferences: {system: boolean, financial: boolean} |

| Ticket | id, subject, requesterId, requester, organizationId, category, priority, status, createdAt, assignedTo, responseHours; messages[{id,senderId,sender,role,message,timestamp}], history[{id,message,timestamp}] |

| Admin | id, name, email, role, status, lastActive, invitationPending, overrides; overrides map permission IDs to booleans |

| Settings | platformName, supportEmail, currency, recruiterApproval, resumeApproval, autoMask, expireDays, maxFileSize, formats[], creditValue, minimumPurchase, maintenance |

| Audit | id, actorId, action, target, targetType, targetId, timestamp, ip, metadata; fixture metadata includes requestId |

Production additions: document IDs/version IDs, file metadata, authorized download references, stable application/history IDs and real timestamps instead of formatted display dates or relative lastActive strings.

Current enums

| Resource | Values |

| --- | --- |

| Candidate account / resume | Active, Suspended / Pending, Approved, Rejected; older uploads can be Archived |

| Candidate application samples | Interviewing, Offered, Rejected |

| Recruiter | PENDING, VERIFIED, REJECTED |

| Hospital | Active, Inactive |

| Payment | SUCCESS, PENDING, FAILED |

| Plan type / unlock type | Monthly, Annual / Search, Direct, Referral |

| Job | ACTIVE, CLOSED, FLAGGED |

| Notification | SYSTEM, FINANCIAL |

| Ticket status / priority | OPEN, IN_PROGRESS, RESOLVED / HIGH, MEDIUM, LOW |

| Ticket category | Technical, Billing, Account, General |

| Staff status | ACTIVE, INACTIVE, INVITED |

Map enum casing explicitly. Refund lifecycle statuses are not implemented in the UI. If backend moderation uses REMOVED, agree its relationship to FLAGGED/CLOSED.

6. Mutation and business rules

Candidates and recruiters

- Candidate suspension is confirmed and currently one-way. Backend must define effects on sessions, applications and profile visibility.
- Resume decisions are available only while Pending. Rejection requires a nonblank reason up to 1,000 characters; notes allow 2,000 characters.
- Notes currently update context on every keystroke. Agree draft/save behavior; submit notes with review or add a dedicated save operation with failure handling.
- Bind reviews to document versions; reject stale reviews if a new upload or another decision supersedes them.
- Resume approval performs no external credential checks. Define the verification procedure and evidence required.
- Recruiter approval requires three local checklist boxes; rejection requires a reason. These are local review controls, not external verification.
- The UI changes one recruiter, while the previous endpoint proposal verifies an organization. Confirm whether decisions affect one recruiter, an organization or all linked recruiters.

Plans, payments, credits and revenue

- Plan name, features and support are required; type is Monthly/Annual. Price is non-negative with at most two decimal places; credits are a non-negative safe integer. Ignore client-supplied statistics.
- Archive prevents future purchases while preserving subscriptions. Plan edits must not rewrite purchase snapshots or existing entitlements.
- Financial pages use INR. Agree decimal strings or integer minor units; map them at the transport boundary. The settings currency selector does not implement currency conversion.
- Refund UI permits successful payments only, a positive amount no greater than the original payment, two decimal places, and a nonblank reason up to 1,000 characters. Backend must also enforce prior-refund accounting, remaining refundable balance, provider eligibility, authorization and concurrency.
- Refunds and payment-derived credit grants require idempotency. Verify provider webhooks before granting credits. Agree treatment of refunds when credits have already been spent.
- Maintain an atomic credit ledger linked to purchases/unlocks/refunds; never trust submitted balances or entitlements.
- Replace sample payment timelines with provider events. Invoice and receipt actions need real document generation and delivery outcomes.

- Revenue must derive from verified successful/settled payments. Agree gross/net revenue, refunds, tax, reporting timezone, comparison periods and subscription counts.

Proposed revenue shape:

```
{
"currency": "INR",
"totalRevenue": "450000.00",
"series": [{ "date": "2026-09-01", "amount": "450000.00" }],
"byPlan": [{ "plan": "Starter", "orders": 45, "amount": "81000.00" }],
"activeSubscriptions": 156,
"averagePerRecruiter": "2884.62",
"monthOverMonthPercent": 7.14
}
```

Current historical revenue covers April-September 2024 using month-start aggregates. Partial-month calculations belong to the backend. Dashboard chart fixtures use lakh-scaled values; explicitly convert real amounts. Historical subscription counts are fixture snapshots.

Jobs, support and notifications

- Jobs can be flagged with a nonblank reason or closed with confirmation. Preserve applications. Reopening is not implemented.
- Ticket replies are trimmed and limited to 1-4,000 characters. Server must supply sender identity, stable IDs and timestamps.
- Assignment accepts active admins with support permission or unassigned; agree mapping from the UI's empty string to the server representation.
- Validate priority/status transitions. Resolution is confirmed from both the action button and dropdown. Persist conversation history separately from authoritative audit events.
- Define response-time/resolution metrics server-side. Notification read state/preferences are per authenticated administrator; preferences do not hide historical alerts or constitute message delivery.

Staff and permissions

Current role defaults are design fixtures, not an approved production policy:

| Role | Permission IDs |

| --- | --- |

| SUPER_ADMIN | users, resumes, payments, jobs, support, settings |

| VERIFICATION_LEAD | users, resumes, payments, jobs, support |

| SUPPORT_LEAD | users, jobs, support |

| FINANCE_ADMIN | resumes, jobs, support |

Finance Admin currently lacks payments permission: confirm explicitly. The role cards omit Support Lead although the role exists. Overrides add/remove permissions from a base role.

Invitations require name, valid email, known role and at least one valid permission. Enforce case-insensitive email uniqueness across active/inactive/invited staff. Generate expiring single-use setup links; activation follows backend acceptance, not a local status change. No initial password is collected.

Demo safeguards block self-deactivation/own-role changes and Super Admin downgrades/overrides. Changing another base role clears overrides. Confirm server policy, including last-active-Super-Admin protection. Deactivation preserves history and must revoke access. Never authorize with submitted actor IDs or the john-admin fixture. Client permission checks cover only some operations; enforce every read/write/export/document access server-side.

Settings and audit

All tabs share one draft and saved settings object. Email exposes supportEmail; Security exposes moderation switches, not an OTP/session policy editor. Maintenance confirmation approves the draft; Save Changes commits it locally. Define actual server maintenance enforcement, Super Admin access and existing-session handling.

expireDays/minimumPurchase are non-negative integers. maxFileSize is positive MB; creditValue is non-negative. File size and credit value allow two decimal places. Demo numeric values are capped at 100,000,000; confirm production limits and units. Formats require at least one of PDF/DOC/DOCX; currency choices are INR/USD/EUR. Backend storage/access services must enforce file limits, formats, expiry and masking.

Audit records are read-only. Emit authoritative events after committed candidate/recruiter/plan/payment changes and JOB_FLAGGED, JOB_CLOSED, TICKET_REPLIED, TICKET_RESOLVED, ADMIN_INVITED, ADMIN_ROLE_CHANGED, ADMIN_DEACTIVATED, SETTINGS_UPDATED, MAINTENANCE_TOGGLED. Cover assignment, priority and permission changes too. Record server-derived actor, target, timestamp, request ID and appropriate change metadata; exclude credentials and reset grants.

7. Shared API conventions to agree

Recommended list response (not currently consumed):

```
{
"data": [],
"pagination": { "page": 1, "pageSize": 20, "total": 0 }
}
```

Use one-based pages, bounded page sizes, deterministic ordering and totals after filtering. UI page sizes vary (candidates 8, pending resumes 6). Metrics/sidebar counters must cover the full authorized dataset, not only loaded rows. Agree which aggregates follow active filters; current pages differ.

Recommended error envelope:

```
{
"error": {
"code": "VALIDATION_ERROR",
"message": "Please check the submitted values.",
"fields": { "price": "Use at most two decimal places." },
"requestId": "server-request-id"
}
}
```

Agree 400/422 validation, 401 absent/expired session, 403 denied permission, 404 missing record, 409 stale/conflicting update, 429 throttling with retry information, and safe 5xx errors. Return canonical committed records after mutations. Use versions/conditional updates for concurrent reviews, settings and staff edits. Do not automatically retry non-idempotent writes.

Use ISO 8601 instants with timezone for events; auth adapter expiry fields explicitly use epoch milliseconds. Date filters use YYYY-MM-DD; agree inclusive end dates and timezone conversion. Audit currently displays IST, exports ISO timestamps and filters UTC calendar dates.

Serve real files through authorized streaming or short-lived signed URLs with filename, MIME type, size and version metadata. Define DOC/DOCX preview conversion; the current resume preview is HTML sample content.

CSV exports include every filtered local row across pages and protect against formula injection. Preserve this with server exports or complete filtered retrieval after introducing pagination. Agree candidate, recruiter, hospital, unlock, payment, job, audit and revenue export contracts. Apply the same permissions to downloads and exports as to the source records.

8. Backend deliverables and open decisions

Provide:

1. OpenAPI specification or equivalent agreed request/response examples, enums, error codes, filters, pagination and sorting.
1. Development/staging API URLs, allowed frontend origins, test accounts for each agreed role and representative test records.
1. Session lookup/logout plus working auth/recovery, followed by read/detail APIs and authorized file access.
1. Persistent mutations, role enforcement, invitation delivery, concurrency controls and audit events.
1. Payment-provider sandbox integration, refund/credit accounting, invoices, receipts and full filtered exports.

Resolve before integration:

- Account-role/admin-level mapping, session mechanism, role matrix and maintenance access.
- Recruiter-versus-organization verification scope; resume verification evidence, versioning and note persistence.
- Money representation, refund/credit rules, reporting definitions and timezone.
- Invitation/override payloads; notification read/preferences; ticket assignment/priority; credit-log, document, invoice, receipt and export contracts.

- Job detail lookup and moderation enums; aggregate filter semantics and list ordering.

Frontend integration status (Revision 3): API reads and writes, loading and error states, silent session refresh, sign-out, the credit-adjust panel and the broadcast panel are built. Remaining work: confirm DTO and enum mapping with the backend, bind permissions from the real session, and confirm the proposed routes listed in section 23.

9. Acceptance checklist

- [ ] Login, refresh, expiry, logout and direct admin URLs use real sessions; unauthorized backend requests fail.
- [ ] Recovery enforces expiry, resend/attempt limits and single-use reset grants without revealing account existence.
- [ ] Every screen handles server data, empty lists, errors and missing records, including direct detail-page refreshes.
- [ ] Filtering, pagination, counters and exports remain correct beyond one page.
- [ ] Mutations persist after refresh; failed writes do not show false success.
- [ ] Concurrent decisions and duplicate refunds/credit grants cannot silently apply twice.
- [ ] Real documents and provider timelines replace samples; file/export access is authorized.
- [ ] Invitations/receipts report actual queue or delivery outcomes; ticket replies use server identity.
- [ ] Roles, deactivation and maintenance rules are enforced server-side and audited.
- [ ] Payments, credits, refunds and revenue reconcile with provider-confirmed records.
- [ ] Production CORS/session configuration and BrowserRouter rewrites are verified.

Existing frontend regression commands:

```
npm run build

npm run test:e2e
```

Build first: Playwright uses installed Microsoft Edge and starts development/production servers on ports 5173/4173. Existing suites: tests/auth.spec.js, tests/admin.spec.js, tests/commerce.spec.js, tests/operations.spec.js, tests/production.spec.js. These test demo behavior and must be adapted or supplemented for real integration; they do not validate a backend.

1. Revision 2 Change Log (7 October 2026)

| Area | Change | Sections |

| --- | --- | --- |

| Job approval | Added PENDING_APPROVAL, ACTIVE, REJECTED, CLOSED, REMOVED lifecycle; admin approve/reject; candidates only see ACTIVE jobs. | 9, 21 |

| Applicants | Recruiters list applicants per job and update application status (Recruiter Handoff section 7). | 9 |

| Notifications | Defined events, GET /notifications and in-app/email channels for admin decisions. | 10.1 |

| Resume rules | rejectionReason returned to candidates; only APPROVED resumes are searchable, unlockable or usable for applying. | 5.2 |

| Plans | planId filter, currentPlan, purchasedPlans, per-organization plan history, subscriber counts. | 6.3, 8.2 |

| Organization | Rejected organizations can resubmit; admin sees resubmittedAt. | 6.2 |

| Conflicts resolved | Resume review route is POST /admin/resumes/{resumeId}/review and organization verification is POST /admin/organizations/{organizationId}/verify (both match the frontend code). Payment status values are uppercase SUCCESS, PENDING, FAILED, REFUNDED in all modules. Error body is { error: { code, message, fields } } in all modules. | 6.2, 5.2, 15 |

25. Revision 3 â€” Changes and additions

25.1 Password recovery follows the shared contract

The earlier adapter expected challengeId, expiresAt and resendAt, which the shared auth endpoints never return, so recovery could not work. The admin portal now calls the shared endpoints exactly.

| Step | Request | Response |

| --- | --- | --- |

| Request code | POST /auth/forgot-password { email } | 202 { message } always, whether or not the account exists |

| Verify code | POST /auth/verify-code { email, code, purpose:"password_reset" } | 200 { verified:true, purpose, resetToken } |

| Reset password | POST /auth/reset-password { email, resetToken, newPassword } | 200 { message }; the token is single use; existing sessions are revoked |

25.2 Session lifecycle

Login stores the access token and refresh token for the browser session only (sessionStorage). When a request returns 401 the portal calls POST /auth/refresh once and repeats the request; if that fails it clears the session and opens /login?expired=1 with a notice. The header now has a Sign out button that calls POST /auth/logout and clears the session; previously the portal had no sign-out. Recommended values for admins: access token 15 minutes, refresh token 8 hours, refresh-token rotation with reuse detection (Auth Handoff section 6).

25.3 Organization verification documents

GET /admin/organizations/{organizationId}/documents

| 200 OK Cache-Control: no-store { "items": [{ "id":"doc_81", "type":"REGISTRATION_CERTIFICATE", "fileName":"certificate.pdf", "mimeType":"application/pdf", "sizeBytes":182044, "createdAt":"2026-10-08T09:30:00Z", "downloadUrl":"https://api.medhire.example/api/files/<signed-token>" }] } |

| --- |

Permission: recruiters.verify. Each download link expires after 5 minutes and is served with Cache-Control private, no-store and X-Content-Type-Options nosniff; an expired or tampered link returns 403 LINK_EXPIRED. Every call writes an audit entry (ORGANIZATION_DOCUMENTS_VIEWED) with the admin id, organization id and IP address. The Recruiter Verification page lists these files; recruiters upload them as described in Recruiter Handoff section 17.3.

25.4 Support tickets from the Contact form

POST /support/tickets is public (login optional) and creates a ticket with source CONTACT_FORM, or IN_APP when the caller is logged in. Body: { name, email, phone?, requesterRole: candidate | recruiter | other, subject, message }. Validation: name 2 to 100 characters, valid email, subject 3 to 150, message 10 to 2000. Limit to 5 requests per hour per IP address; a hidden website field is a spam trap, and when it is filled nothing is stored. Each ticket also creates an admin notification of type SUPPORT_TICKET_NEW linking to /admin/support-tickets/{id}.

25.5 Candidate self-service export and deletion

Candidates can export their data and delete their account (Candidate Handoff, Appendix F). For admin screens this means: deleted candidates appear as â€œDeleted userâ€□ with status DELETED and no contact details, their resumes disappear from the moderation queue, existing applications and unlock records remain, and two audit actions appear in the log: CANDIDATE_DATA_EXPORTED and CANDIDATE_ACCOUNT_DELETED (neither stores personal data in its metadata).

25.6 Screens now built

| Screen | Who may use it | Contract |

| --- | --- | --- |

| Adjust credits (Hospital Plans page) | SUPER_ADMIN, FINANCE_ADMIN | POST /admin/organizations/{organizationId}/credits/adjust { delta, reason } with an Idempotency-Key header. delta is a non-zero whole number (negative removes credits); reason is required, at least 5 characters, and is written to the audit log. A repeated Idempotency-Key must not apply twice. |

| Send broadcast (Notifications page) | SUPER_ADMIN, OPS_ADMIN | POST /admin/notifications/broadcast { audience, title, message }. audience: ALL_RECRUITERS, ALL_CANDIDATES or ALL_USERS (the last two are new in Revision 3). title up to 120 and message up to 1000 characters. The UI asks for confirmation. Rate-limit and audit-log every send. |

The portal hides these panels for other permission levels; the backend must still re-check the level on every request.

25.7 Revision 3 acceptance checklist

[ ] Forgot password, verify code and reset password work end to end using only the shared auth endpoints.

[ ] Letting the access token expire does not interrupt an admin who keeps working; a revoked refresh token returns to login with a notice.

[ ] Sign out invalidates the refresh token on the server.

[ ] Organization documents open only through signed links that expire after 5 minutes, and each view is audit-logged.

[ ] Contact-form submissions appear as support tickets and notifications; the spam trap stores nothing.

[ ] A credit adjustment with a repeated Idempotency-Key applies once.

[ ] Broadcasts are limited to SUPER_ADMIN and OPS_ADMIN and appear in the audit log.

Cross-portal decisions required before backend implementation

1. Confirm canonical API base URL, path prefixing, allowed frontend origins, credentialed CORS policy, and staging URLs. Avoid double `/api` prefixes: each frontend's base URL convention must be documented alongside relative endpoint paths.
2. Publish canonical role names and session DTOs for candidate, recruiter, and admin. Confirm access-token and refresh-token storage/rotation, expiry, refresh reuse detection, logout/revocation, remember-me lifetime, and CSRF policy if cookies are used.
3. Publish common response envelopes, pagination defaults/caps, stable sorting, filter semantics, timestamp/timezone rules, and a consistent validation/error envelope with field errors and request IDs.
4. Confirm authorization and ownership on the server for every resource, mutation, export, file and download. Resolve organization approval gates and admin role/permission matrix.
5. Confirm candidate resume/profile/document privacy and approval policy; recruiter organization verification scope; admin document review versions and audit requirements.

6. Agree money representation, currencies, payment settlement and webhook verification, idempotency, refund accounting, credit ledger consistency, invoice delivery, and revenue definitions.
7. Confirm persistent state transitions and audit events for onboarding, job moderation, candidate applications, notifications, support, staff/roles, settings, and account deletion/export.
8. Provide OpenAPI (or equivalent), staging credentials and representative records, file upload limits, and end-to-end acceptance criteria. No sample identity or frontend-only result should be treated as production data.

Suggested integration order

1. Shared auth/session lifecycle and role/organization state.
2. Candidate onboarding persistence and secure resume/photo uploads.
3. Recruiter organization review and approved workspace read paths.
4. Candidate discovery, privacy masking, unlock ledger, and shortlists.
5. Recruiter checkout/webhook and payment history.
6. Admin read/detail APIs, secure files, then auditable mutations and permission enforcement.
7. Cross-portal reconciliation, exports, observability, and production CORS/session configuration.